

১. Closure কী? (এক লাইনের সংজ্ঞা)

Closure হলো এমন একটি **function**, যা তার **outer scope**-এর **variable**-গুলোকে **outer function** শেষ হয়ে যাওয়ার পরও **access** করতে পারে।

২. Closure আর Normal Function-এর মধ্যে পার্থক্য কী?

Normal Function	Closure
শুধু নিজের local variable নিয়ে কাজ করে।	নিজের local variable-এর পাশাপাশি outer scope-এর variable-ও মনে রাখতে পারে।
Function শেষ হলে local variable নষ্ট হয়ে যায়।	Outer function শেষ হলেও প্রয়োজনীয় variable ধরে রাখে।
State ধরে রাখতে পারে না।	State ধরে রাখতে পারে।

উদাহরণ:

```
// Normal Function
function normal() {
  let x = 10;
  console.log(x);
}

// Closure
function outer() {
  let x = 10;

  return function () {
    console.log(x);
  };
}
```

৩. Closure দিয়ে Private Variable কীভাবে বানাবেন?

```
function counter() {
  let count = 0; // Private Variable

  return {
    increment() {
      count++;
    },
  };
}
```

```
    getCount() {  
      return count;  
    }  
  };  
}
```

```
const c = counter();
```

```
c.increment();  
c.increment();
```

```
console.log(c.getCount()); // 2
```

এখানে

count

বাইরে থেকে access করা যাবে না।

```
console.log(c.count); // undefined
```

শুধু `increment()` এবং `getCount()`-এর মাধ্যমে ব্যবহার করা যাবে।

এটাই Private Variable।

8. Loop-এ **var** ব্যবহার করলে **setTimeout()** একই **output** দেয় কেন?

```
for (var i = 1; i <= 3; i++) {  
  setTimeout(() => {  
    console.log(i);  
  }, 1000);  
}
```

Output:

4
4
4

কেন?

- **var** হলো **function-scoped**।
- পুরো loop-এর জন্য একটাই **i variable** থাকে।
- **setTimeout()** callback পরে execute হয়।
- ততক্ষণে loop শেষ হয়ে **i = 4** হয়ে গেছে।

- Closure সেই একই `i`-কে refer করে।

তাই তিনবারই `4` print হয়।

`let` ব্যবহার করলে:

```
for (let i = 1; i <= 3; i++) {  
  setTimeout(() => console.log(i), 1000);  
}
```

Output:

1
2
3

কারণ `let` প্রতিটি iteration-এর জন্য নতুন binding তৈরি করে।

৫. Closure কি Memory Leak তৈরি করতে পারে? কেন?

উত্তর: হ্যাঁ, পারে।

কেন?

Closure যেসব variable ব্যবহার করে, JavaScript সেগুলোকে memory-তে ধরে রাখে।

যদি closure অপ্রয়োজনীয় বড় data-কে দীর্ঘ সময় ধরে reference করে রাখে, তাহলে Garbage Collector সেই memory মুক্ত করতে পারে না।

উদাহরণ:

```
function outer() {  
  const hugeArray = new Array(1000000).fill("data");  
  
  return function () {  
    console.log(hugeArray.length);  
  };  
}
```

```
const fn = outer();
```

এখানে `hugeArray`-কে `fn` closure reference করছে।

যতক্ষণ `fn` জীবিত থাকবে, ততক্ষণ `hugeArray`-ও memory-তে থাকবে।

এটি সবসময় **memory leak** নয়, কিন্তু যদি আর দরকার না থাকা সত্ত্বেও closure-এর reference রেখে দাও, তাহলে অপ্রয়োজনীয় memory দখল হয়ে থাকতে পারে এবং memory leak-এর মতো সমস্যা তৈরি হতে পারে।

Interview-তে ৩০ সেকেন্ডের উত্তর

- **Closure:** Function outer scope-এর variable outer function শেষ হওয়ার পরও access করতে পারে।
- **Difference:** Normal function state ধরে রাখে না; closure outer variable মনে রাখে।
- **Private Variable:** Variable-কে outer function-এর ভিতরে রেখে inner function দিয়ে access করাই private variable।
- **var + setTimeout:** var function-scoped হওয়ায় সব callback একই variable share করে; loop শেষে তার final value-ই print হয়।
- **Memory Leak:** হতে পারে, যদি closure অপ্রয়োজনীয় object/variable-কে reference করে রাখে, ফলে Garbage Collector memory free করতে না পারে।

What is a Callback Function?

A callback function is a function that is passed as an argument to another function and is executed later by that function.

In simple words:

A callback is a function that you give to another function so it can call it when needed.